

ProfitPilot

Project Workflow & Build Timeline

Stateful Multi-Agent Business Intelligence System

Phase 1: Resume Build • 6 Weeks • Version 1.0

PHASE 1

Resume Build — Weeks 1-6

PHASE 2

MVP Launch — Months 3-6

PHASE 3

Scale — Month 6+

1. Project Overview

ProfitPilot is a Stateful Multi-Agent Orchestration system that fuses structured Excel/Tally financial data with PDF business strategy documents to produce verified, hallucination-free business insights. This document defines the exact workflow, daily tasks, and milestones for the 6-week Phase 1 build.

💡 *Build order is non-negotiable. Each week depends on the previous. Do not start Week 3 until Week 2's agent is working end-to-end.*

1.1 The Core Workflow (How ProfitPilot Works)

Step	What Happens	Agent Responsible	Output to State
1	User uploads Excel + PDF	UI Layer	raw_files
2	Excel parsed, schema extracted, data audited	Excel Engine	data_schema
3	PDF parsed, chunked, embedded into ChromaDB	RAG Engine	business_goals
4	User asks a question	UI Layer	user_query
5	Query classified: QUANTITATIVE / QUALITATIVE / HYBRID	Supervisor	query_type
6	Python code written + executed in secure REPL	Data Analyst	analyst_result
7	ChromaDB queried, strategy passages retrieved	Strategic Consultant	rag_context

8	Data result + strategy context fused into insight	Strategic Consultant	profit_insight
9	Numbers verified, hallucinations checked	Critique Agent	critique_passed
10	Verified insight rendered to user	UI Layer	final_output

1.2 The Business State Object

Every agent reads from and writes to one shared state object. Define this first — it is the contract between all agents.

Field	Type	Set By	Used By
data_schema	dict	Excel Engine	Data Analyst, Supervisor
business_goals	str	RAG Engine	Strategic Consultant, Supervisor
user_query	str	UI Layer	Supervisor, all agents
query_type	enum	Supervisor	LangGraph router
analyst_code	str	Data Analyst	Critique Agent
analyst_result	dict	Data Analyst	Strategic Consultant, Critique
rag_context	list[str]	Strategic Consultant	Critique Agent
profit_insight	str	Strategic Consultant	Critique Agent
critique_passed	bool	Critique Agent	Supervisor (retry logic)
final_output	str	Critique Agent	UI Layer
error_log	list	Any agent	Supervisor (retry routing)

2. Six-Week Build Timeline

Week 1 Foundation	Week 2 Analyst Agent	Week 3 RAG Pipeline	Week 4 Supervisor + Critique	Week 5 Streamlit UI	Week 6 Polish & Deploy
----------------------	-------------------------	------------------------	------------------------------------	------------------------	---------------------------

WEEK 1 Foundation

state.py + Excel Engine + environment setup

- Set up Python environment: pip install langgraph langchain chromadb streamlit pandas openpyxl pymupdf fpdf2
- Create state.py — define BusinessState as a TypedDict with every field. This is the contract. Do not skip fields.
- Build Excel Engine: file upload handler, Pandas schema extractor, Data Audit (missing values, outliers, dtypes)
- Download Superstore Sales dataset from Kaggle. Run your audit against it.
- Write your demo strategy PDF (1 page, 3 goals with specific numbers — e.g. 'Grow West region revenue by 15%')
- Verify: can you print a clean data_schema dict from the Superstore file?

DELIVERABLES: ✓ state.py with all fields ✓ Excel Engine working ✓ Real dataset loaded

WEEK 2 Data Analyst Agent

Code-writing + REPL execution + retry loop

- Build the code-writing prompt: system prompt tells Claude to write Pandas code, never guess math, always use df variable
- Build the secure REPL executor: subprocess with 10-second timeout, capture stdout and stderr
- Build retry logic: parse error message, include it in next prompt, retry up to 3 times before escalating
- Write analyst_code and analyst_result into BusinessState after each successful execution
- Test with 10 natural language questions — at least 3 must involve multi-step calculations
- Key test: 'What are the top 5 products by revenue in the West region this year?' must return correct numbers

DELIVERABLES: ✓ 10 NL queries answered correctly ✓ Retry loop working ✓ analyst_result in state

⚠ *Do not proceed to Week 3 until the Data Analyst agent correctly answers all 10 test questions. The RAG pipeline depends on analyst_result being reliable.*

WEEK 3 RAG Pipeline

PDF ingestion + ChromaDB + Strategic Consultant

- Build PDF ingestion: PyMuPDF text extraction, 512-token chunks with 50-token overlap
- Embed chunks with text-embedding-3-small and store in ChromaDB collection
- Extract business_goals summary from PDF on upload — write to BusinessState
- Build Strategic Consultant agent: retrieves top-5 relevant chunks, fuses with analyst_result

- Fusion prompt template: 'Data shows X. Strategy says Y. The tension/alignment is Z. Recommendation: W.'
- Key test: 'Does our top-selling product align with our stated 2026 expansion strategy?' must produce a coherent fused insight

DELIVERABLES: ✓ PDF indexed in ChromaDB ✓ Strategic Consultant working ✓ Fusion insight produced

WEEK 4 Supervisor + Critique Agent

LangGraph graph + query routing + hallucination check

- Build Supervisor agent: classify query as QUANTITATIVE, QUALITATIVE, or HYBRID using a simple prompt
- Wire the LangGraph StateGraph: define nodes (supervisor, analyst, consultant, critique), edges, and conditional routing
- Build Critique Agent: extract all numbers from profit_insight, verify each against analyst_result
- Build Critique Agent source check: flag any claim not traceable to Excel data or PDF passages
- If critique fails: Critique writes failure reason to error_log, graph routes back to Supervisor
- Test the retry loop deliberately: give the consultant a prompt that produces a hallucinated number, confirm Critique catches it
- Key test: end-to-end query must complete in under 30 seconds on your machine

DELIVERABLES: ✓ Full LangGraph graph wired ✓ Critique catches 1+ hallucination ✓ Retry loop tested

💡 *The Critique Agent catching a hallucination in your demo is more impressive than perfect output. Show the QC loop working — interviewers remember it.*

WEEK 5 Streamlit UI

Three-tab interface + file upload + report generation

- Build Tab 1 (Chat): file upload for Excel + PDF, chat input wired to LangGraph graph, response rendered with source citations
- Build sidebar: Data Audit summary, schema preview, model token usage counter
- Build Tab 2 (Explorer): PyGWalker embedded with the uploaded DataFrame — zero extra config needed
- Build Tab 3 (Report): FPDF2 generates a 2-page Profit Roadmap with top 3 insights, key numbers, and recommendations
- Wire session state: uploaded files and conversation history persist across tab switches
- Test the full user journey: upload files, ask 3 questions, switch to Explorer, download report

DELIVERABLES: ✓ Chat tab working ✓ PyGWalker tab working ✓ PDF report downloadable

WEEK 6 Polish & Deploy

Real demo data + README + Streamlit Cloud deployment

- Replace any placeholder data with the Superstore dataset — run 5 demo questions and save their outputs
- Write GitHub README: project overview, architecture diagram link, setup instructions, 3 design decision explanations
- Add architecture diagram to README (use draw.io or Excalidraw — export as PNG)
- Deploy to Streamlit Cloud: connect GitHub repo, set API keys as secrets, verify deployment
- Record a 3-minute Loom video: 60 seconds on the problem, 90 seconds on the demo, 30 seconds on architecture decisions
- Prepare your 3 interview answers: why LangGraph, why REPL, how Critique Agent works — each under 60 seconds

DELIVERABLES: ✓ Deployed on Streamlit Cloud ✓ README with architecture ✓ Demo video recorded

3. Recommended Daily Schedule

Each week has 5 working days. This schedule assumes 3-4 focused hours per day. Adjust if you have more time, but keep the weekly deliverable gates.

Day	Focus	Time	Done When...
Day 1 (Mon)	Setup + state.py	2-3 hrs	BusinessState TypedDict imports cleanly with no errors
Day 2 (Tue)	Core feature build	3-4 hrs	Primary agent or component runs on a test input
Day 3 (Wed)	Edge cases + error handling	2-3 hrs	Handles at least 2 failure modes gracefully
Day 4 (Thu)	Integration + testing	3-4 hrs	Component writes correct output to BusinessState
Day 5 (Fri)	Review + next week prep	1-2 hrs	Weekly deliverable confirmed. Next week's Day 1 planned.

⚠ *Protect your Day 5 review. The most common failure mode is skipping the weekly gate and carrying technical debt into the next week. It compounds fast.*

4. Agent Interaction Workflow

The following table describes the complete data flow between agents. Every arrow in the LangGraph graph corresponds to a row here.

From	To	Trigger	Data Passed	Failure Path
User	Supervisor	User submits query	user_query	—
Supervisor	Data Analyst	query_type = QUANTITATIVE or HYBRID	user_query + data_schema	If schema missing: prompt re-upload
Supervisor	Strategic Consultant	query_type = QUALITATIVE	user_query + business_goals	If no PDF: prompt upload
Data Analyst	REPL Executor	Code written	analyst_code	Syntax error: rewrite up to 3x
REPL Executor	Data Analyst	Execution error	stderr + traceback	After 3 fails: escalate to Supervisor
REPL Executor	BusinessState	Execution success	analyst_result	—

Strategic Consultant	ChromaDB	RAG retrieval needed	user_query embedding	No results: use business_goals only
Strategic Consultant	BusinessState	Fusion complete	profit_insight + rag_context	—
Critique Agent	BusinessState	Verification passed	critique_passed=True, final_output	—
Critique Agent	Supervisor	Verification failed	error_log with reason	Supervisor re-routes with error context
Supervisor	UI Layer	final_output ready	final_output	—

5. Testing Checklist Per Week

Week 1 — Foundation Tests

- Upload Superstore.xlsx — does data_schema print correctly with all column names and types?
- Upload a broken Excel (merged cells, missing headers) — does the audit report the issues?
- Is business_goals extracted from your demo PDF as a structured summary?

Week 2 — Data Analyst Tests

- 'What is total revenue?' — simple sum, must match manual Excel calculation exactly
- 'Top 5 products by profit margin?' — multi-column calculation, verify against manual check
- 'What was revenue in Q3?' — requires date parsing, test with actual date column
- Deliberately break the query — does the retry loop rewrite code and recover?
- Does analyst_result write to BusinessState correctly after execution?

Week 3 — RAG Tests

- Ask a strategy question — does the retrieved chunk actually come from your PDF?
- 'Does our top product align with our 2026 strategy?' — does the fusion make logical sense?
- Ask about something NOT in the PDF — does the agent say so, or hallucinate?

Week 4 — Supervisor + Critique Tests

- Pure data question — does Supervisor route ONLY to Data Analyst (not consultant)?
- Pure strategy question — does Supervisor route ONLY to Strategic Consultant?
- Manually inject a wrong number into profit_insight — does Critique catch it?
- Trigger a Critique failure — does the graph retry instead of crashing?
- Full end-to-end: upload → ask → response. Time it. Must be under 30 seconds.

Week 5 — UI Tests

- Upload both files, ask 3 questions — does chat history persist across questions?
- Switch between tabs — does state reset or persist correctly?
- PyGWalker: drag a column, create a chart — does it render without error?
- Download the PDF report — does it open, render correctly, and contain real numbers?

Week 6 — Pre-Deploy Tests

- Deploy to Streamlit Cloud — does it load with a fresh browser session (no local state)?
- Run all 5 demo questions on the deployed version — same results as local?
- Test on mobile browser — is the layout usable enough for a quick demo?

6. Interview Preparation

These questions will come up in every technical interview. Prepare these answers before Week 6, not after.

Question	Your Answer (Key Points)
Why LangGraph over plain LangChain?	LangGraph supports cycles — if the Data Analyst's code fails, the graph can retry without restarting. LangChain chains are linear and stateless. ProfitPilot needs retry loops and shared state across agents, which LangGraph's StateGraph provides natively.
Why use a Python REPL instead of asking the LLM to calculate?	LLMs are probabilistic — they can hallucinate numbers, especially in multi-step arithmetic. The REPL executes deterministic Python code, so the output is mathematically guaranteed. This is the 'zero-hallucination math' pattern.
How does the Critique Agent prevent hallucinations?	It extracts every number from the profit_insight narrative using regex, then verifies each against analyst_result from the REPL. Any number that can't be sourced to the Excel data or PDF is flagged. The output only reaches the user if all checks pass.
Why ChromaDB over Pinecone for this project?	ChromaDB runs locally with zero configuration — ideal for Phase 1 development and offline demos. Pinecone is cloud-hosted and production-ready for multi-tenant Phase 2. The embedding model and query interface are identical, so migration is straightforward.
How would you scale this for 1,000 concurrent users?	Replace ChromaDB with Pinecone for multi-tenant vector isolation. Move the REPL executor to a sandboxed container (e.g., AWS Lambda). Add a job queue (Celery + Redis) for async agent execution. The LangGraph graph itself is stateless per session.
What was the hardest part to build?	The Critique Agent's number verification. Extracting numbers from natural language narrative reliably required handling currency formatting, percentages, and ranges. A simple regex wasn't enough — I used a combination of regex and a secondary LLM call to classify whether each number was a claim or context.

7. Resume Bullet Points

Use these directly on your resume, adapting the subtitle based on the role you're applying for.

For AI/ML Engineer Roles

- Built ProfitPilot, a stateful multi-agent orchestration system using LangGraph with 4 specialized agents (Supervisor, Data Analyst, Strategic Consultant, Critique) and a shared TypedDict state object
- Implemented a secure Python REPL executor with 3-attempt retry logic, eliminating LLM arithmetic hallucinations — all numerical outputs are code-verified
- Built a RAG pipeline using PyMuPDF, ChromaDB, and text-embedding-3-small to retrieve and fuse business strategy context with quantitative Excel analysis
- Designed a Critique Agent that performs number grounding — verifying all narrative claims against REPL output before surfacing to the user

For Data Analyst / Data Science Roles

- Built ProfitPilot, an agentic analytics system that answers natural language business questions against Excel datasets using Pandas, verified by a sandboxed REPL
- Implemented automated Data Audit on upload: schema detection, missing value analysis, outlier flagging, and data type inference across messy real-world spreadsheets
- Embedded PyGWalker for interactive drag-and-drop data exploration, enabling manual verification of AI-generated insights
- Generated automated 'Profit Roadmap' PDF reports using FPDF2, combining quantitative findings with strategy-aligned recommendations

For Product Analyst / Associate PM Roles

- Designed and built ProfitPilot, an AI product for MSME business intelligence — defined user journey from file upload to insight delivery with a target Time-to-First-Insight under 30 seconds
- Identified a B2B2C distribution strategy via CA firms for Phase 2 launch, targeting ₹30,000 MRR from 10 pilot customers as the validation threshold
- Defined product success metrics: query accuracy, Critique Agent catch rate, REPL execution success rate, and end-to-end response latency
- Built a three-tab Streamlit product: Chat interface, PyGWalker data explorer, and automated PDF report — designed for non-technical business users

The build order is the strategy.

state.py first. Data Analyst second. Everything else follows.
A working core loop beats a half-finished full system every time.